

Project | IE University — School of Science & Technology

November 7, 2025

Finding the Optimal Flight Route

OptiFly✈️

We all travel, but do we travel in smart way?

We've all spent hours comparing flights, layovers, and prices, time we don't really have.

Now picture 5 cities, 20 days.

How do you see it all without overspending or running out of time?

That's exactly what our model solves.



Current solutions are limited

- One origin, one destination
- Multiple destinations but fix order of cities

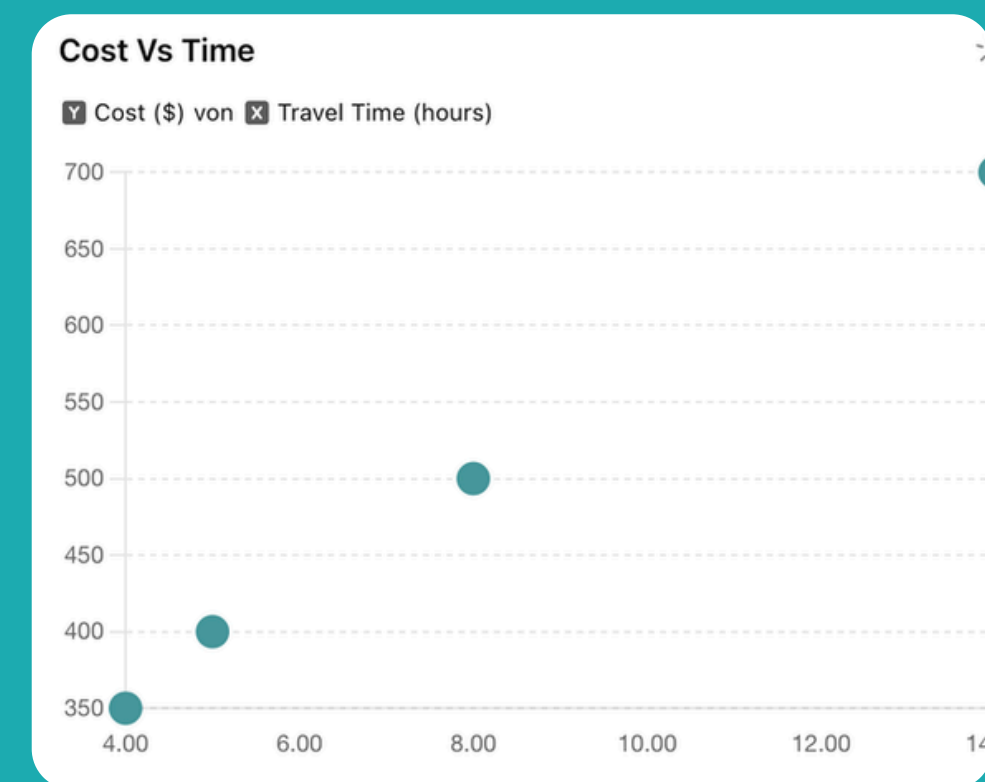


Google Flights

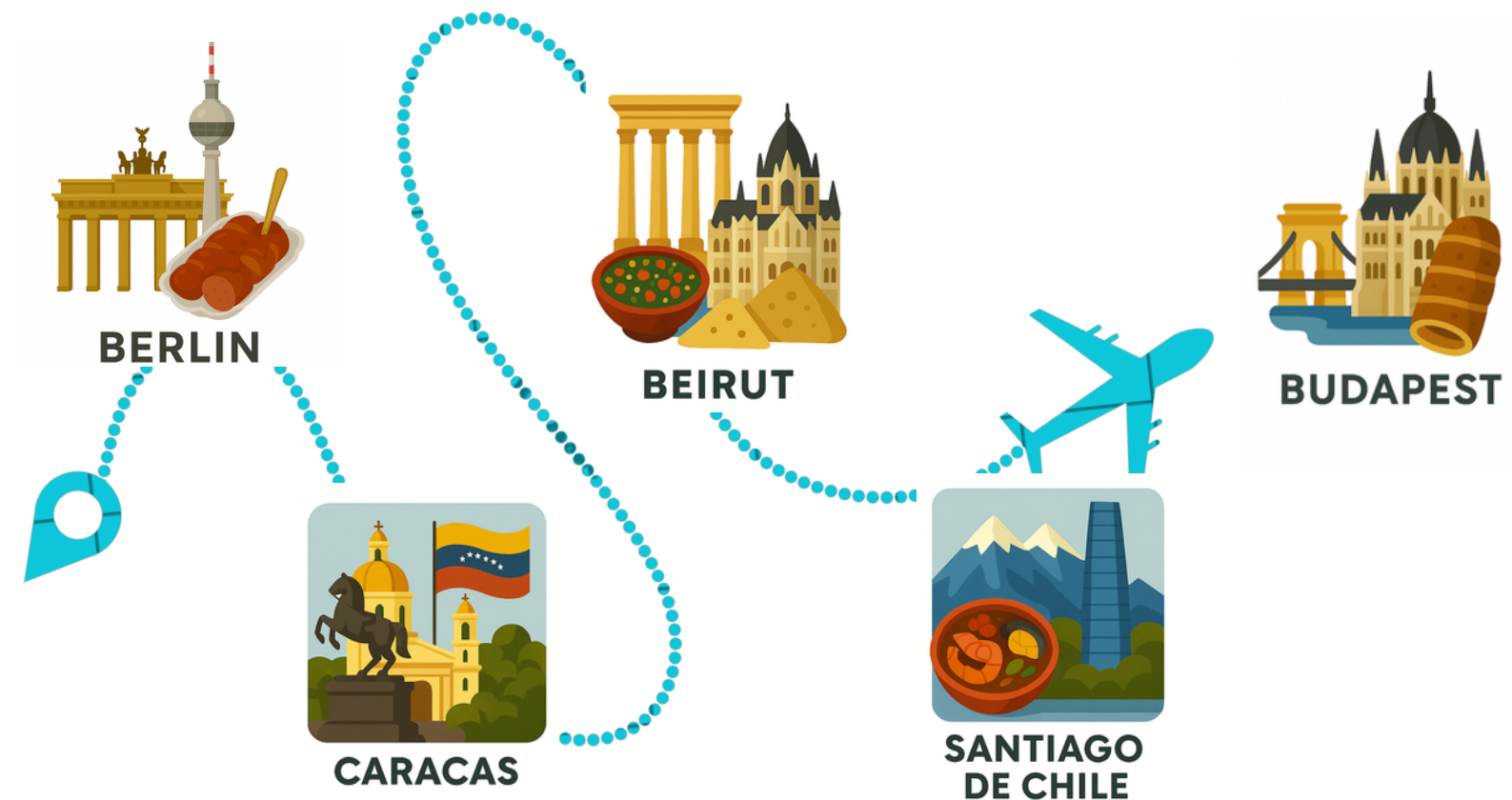
From	To
✈	✈
Departure	Return
DD/MM/YYYY	DD/MM/YYYY
1 adult	Economy
Search flights	

Skyscanner

From	To
✈	✈
Departure	Return
DD/MM/YYYY	DD/MM/YYYY
1 adult	Economy
Search flights	



Our real problem



Smart travel made simple with **OptiFly**

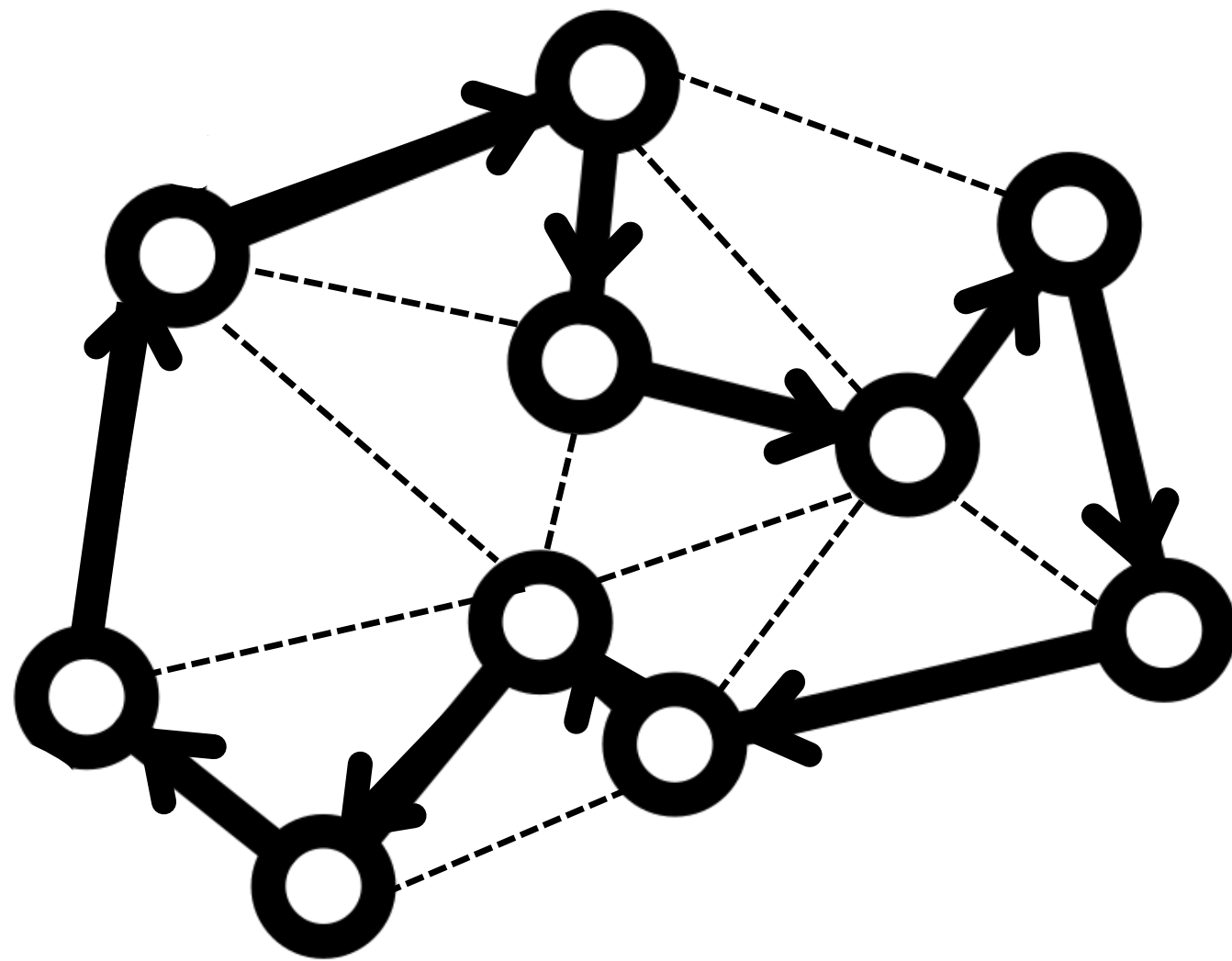
Discover the most efficient and affordable path connecting Berlin, Beirut, Budapest, Caracas, and Santiago de Chile.

We begin our holiday on Dec 19th, from Madrid.

Classic TSP Recap (Meet the Math behind it)



Find the route that **visits every city once**, **returns to the starting point** and **minimizes total flight cost**.



Input:

$N = \{1, 2, \dots, n\}$: set of cities

C_{ij} : cost of traveling from city i to city j

Decision variable:

$x_{ij} \in \{0, 1\}$: 1 if traveling directly from i to j , 0 otherwise.

Restrictions:

Each city is visited exactly once

$$\sum_{\substack{j=1 \\ j \neq i}}^n x_{ij} = 1, \quad \forall i \in N$$

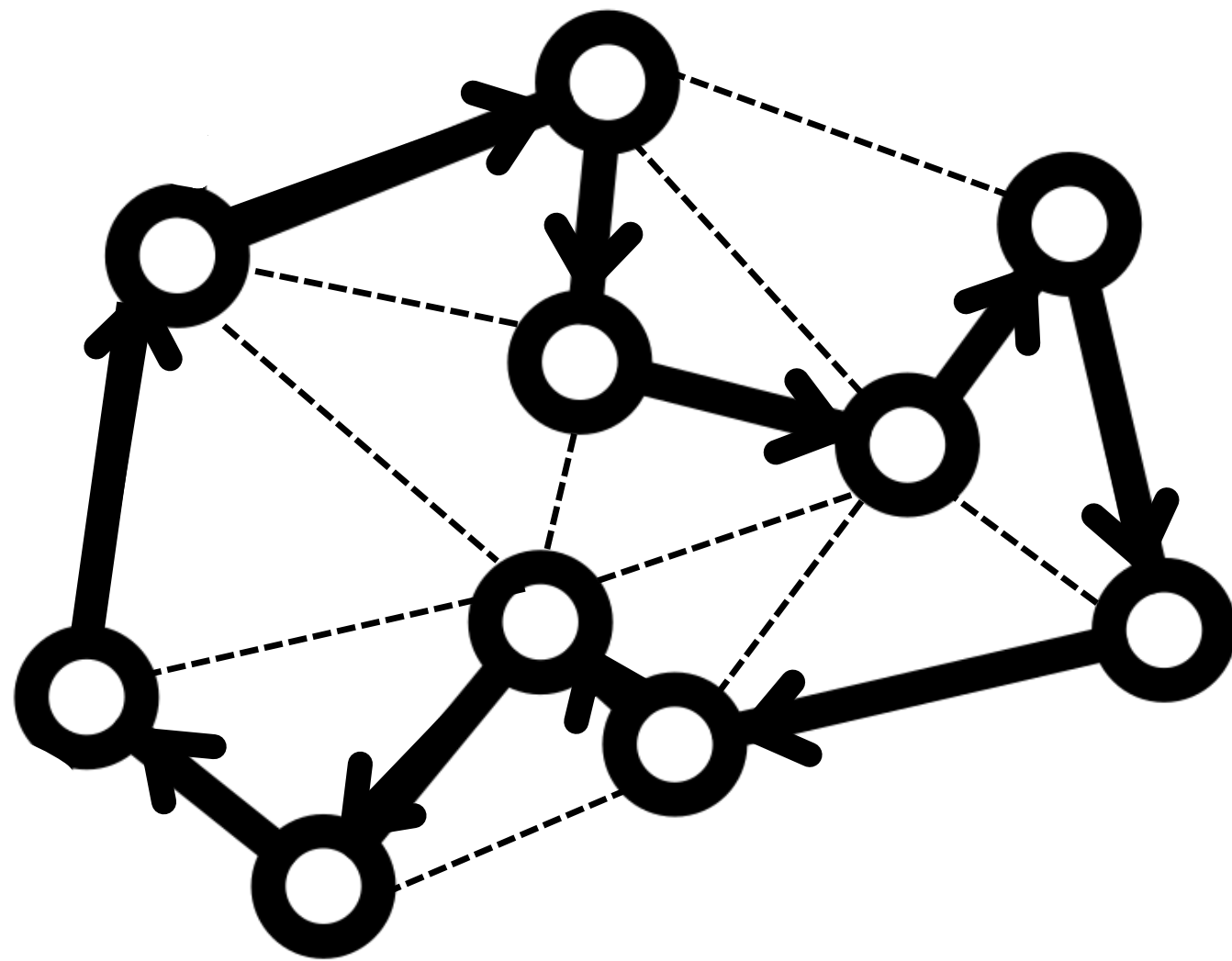
$$\sum_{\substack{i=1 \\ i \neq j}}^n x_{ij} = 1, \quad \forall i \in N$$

$$\min Z = \sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n C_{ij} x_{ij}$$

Variation: TSP with time windows

Find the route that **visits every city once**, **returns to the starting point**, **minimizes total flight cost**, and respects both:

- a fixed travel window T , and
- stay durations ($\pm$ tolerance) in each city.



Input:

$N = \{1, 2, \dots, n\}$: set of cities

C_{ij} : Cost of traveling from city i to city j

NEW:

T_{ij} : Duration of flight from i to j

avg_i : Average desired stay in city i

tol : Tolerance (flexibility) around the average stay

T : Total trip duration

Decision variable:

$x_{ij} \in \{0, 1\}$: 1 if traveling directly from i to j , 0 otherwise.

NEW:

s_i : stay time in city i (number of days or hours spent there)

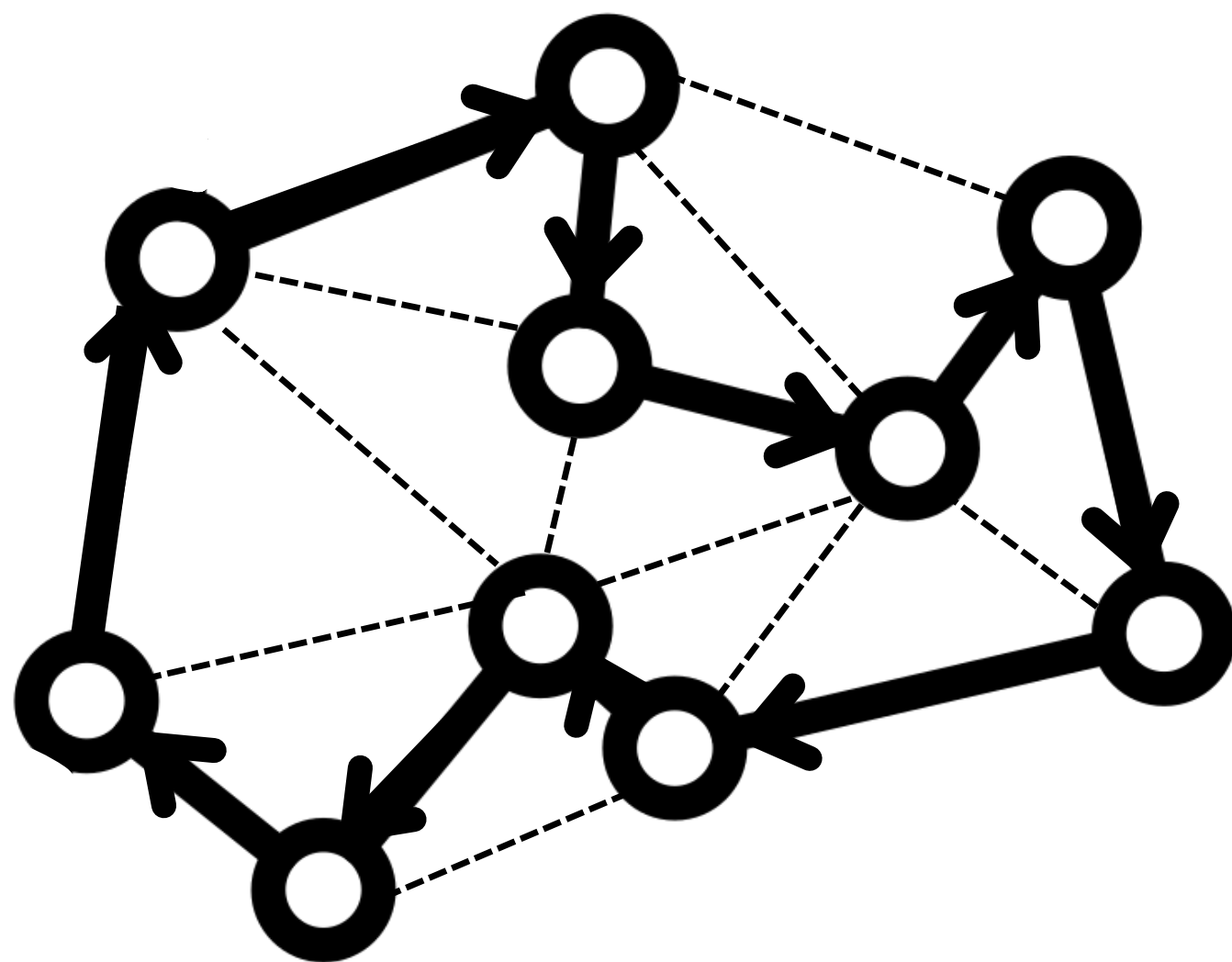
t_i : arrival day at city i

$$\min Z = \sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n C_{ij}(t_i) x_{ij}$$

Variation: TSP with time windows 2

Find the route that **visits every city once**, **returns to the starting point**, **minimizes total flight cost**, and respects both:

- a fixed travel window T , and
- stay durations ($\pm$ tolerance) in each city.



Restrictions:

Each city is visited exactly once

$$\sum_{\substack{j=1 \\ j \neq i}}^n x_{ij} = 1, \quad \forall i \in N$$

$$\sum_{\substack{i=1 \\ i \neq j}}^n x_{ij} = 1, \quad \forall i \in N$$

NEW:

1) *Stay duration range (implicit time window per city)*

$$s_i \in [avg_i - tol, avg_i + tol]$$

2) *Time propagation (arrival times)*

$$t_j \geq t_i + s_i + T_{ij} - M(1 - x_{ij}), \quad \forall i, j \in N$$

where M is a large constant to deactivate the constraint when the flight doesn't exist

3) *Total travel horizon*

$$0 \leq t_i \leq T, \forall i \in N$$

$$\min Z = \sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n C_{ij}(t_i) x_{ij}$$

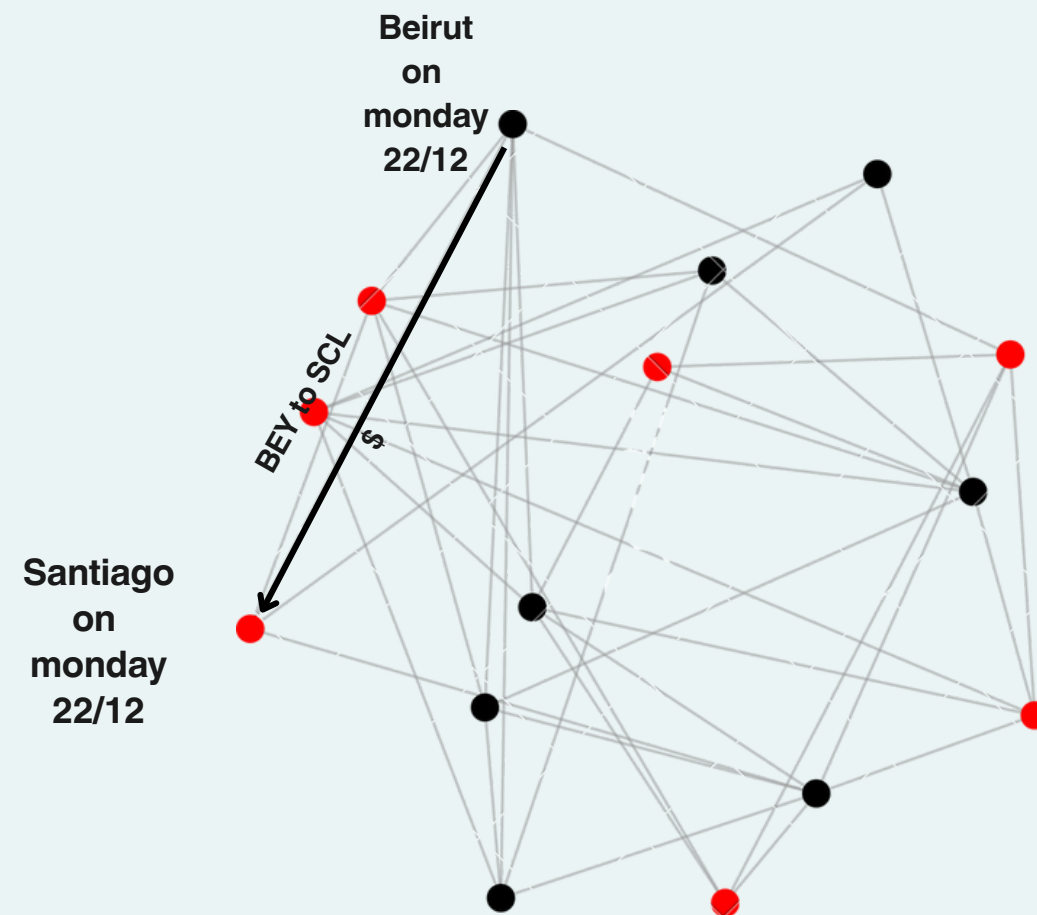
Let's Model it

Understanding essential concepts for analysis

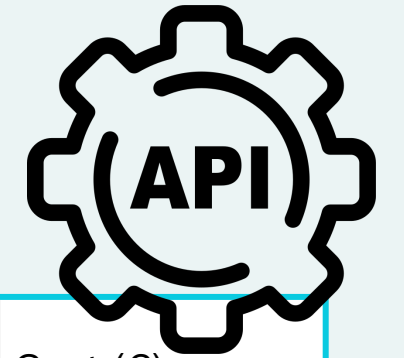
Weighted and directed graph

- **node**: a city on a specific date
- **edge**: flight cost

Each city also gets its own **time window** [$open_i$, $close_i$].



OptiFly^x



Using a global flight API, we collect **cost** $C(i,j)$ and **travel time** $T(i,j)$ for all pairs of cities.

From → To	Departure Date	Arrival Date	Duration	Cost (€)
Paris → Rome	May 3, 2025	May 3, 2025	2h 10m	95
Rome → Athens	May 6, 2025	May 6, 2025	1h 55m	80
Athens → Vienna	May 9, 2025	May 9, 2025	2h 25m	110
Vienna → Prague	May 12, 2025	May 12, 2025	1h 10m	70
Prague → Amsterdam	May 15, 2025	May 15, 2025	1h 35m	120
Amsterdam → Madrid	May 18, 2025	May 18, 2025	2h 35m	140
Madrid → Lisbon	May 20, 2025	May 20, 2025	1h 20m	75
Lisbon → Paris	May 22, 2025	May 22, 2025	2h 25m	105

Held-Karp Algorithm



Dynamic Programming for the Traveling Salesman Problem (Held–Karp)

We already know the Traveling Salesman Problem, try all city orders and pick the cheapest one.

But that's $n!$ possibilities.

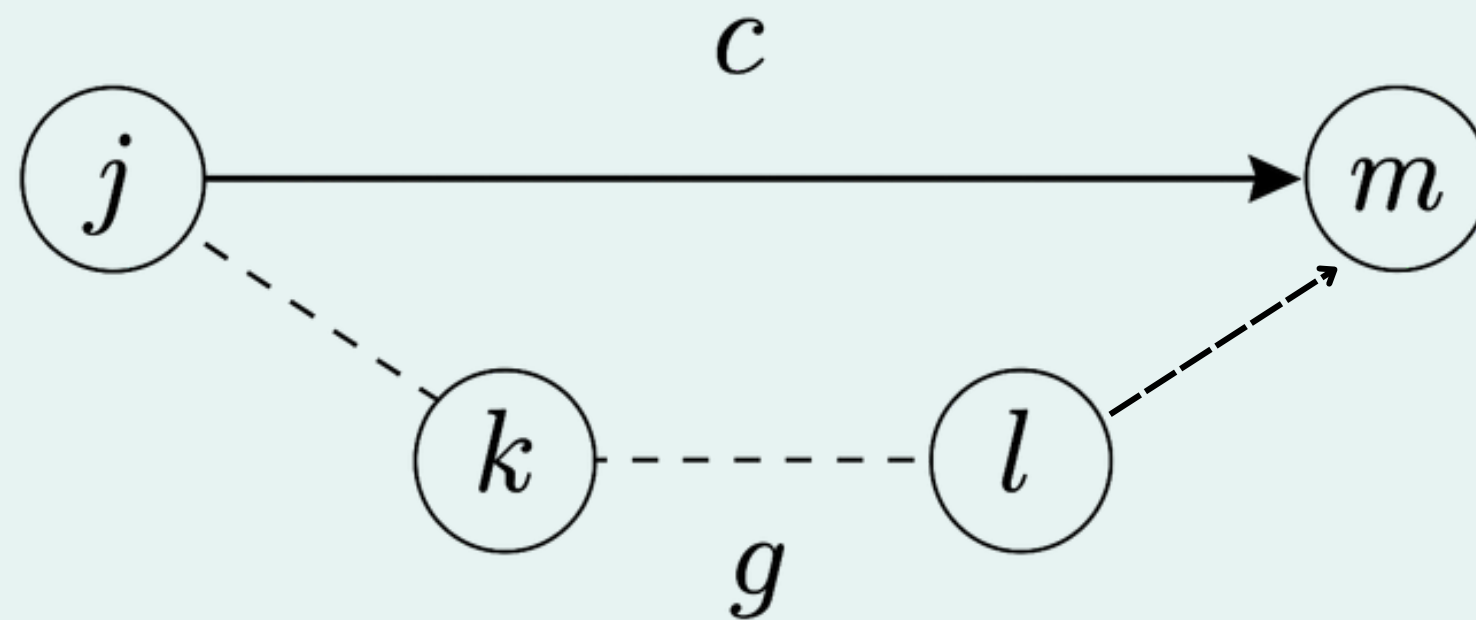
Even for 10 cities, that's over 3 million routes, impossible to check one by one. We use the Held–Karp Dynamic Programming algorithm to shrink that explosion.

Formula

$$DP[S][j] = \min_{i \in S, i \neq j} \{DP[S \setminus \{j\}][i] + C_{ij}\}, \quad \forall S \subseteq N, j \in S, j \neq 1$$

Held-Karp Algorithm

Dynamic Programming for the Traveling Salesman Problem (Held-Karp)



How it works

In this case, we want the cost from $j \rightarrow m$.

There are two possible ways:

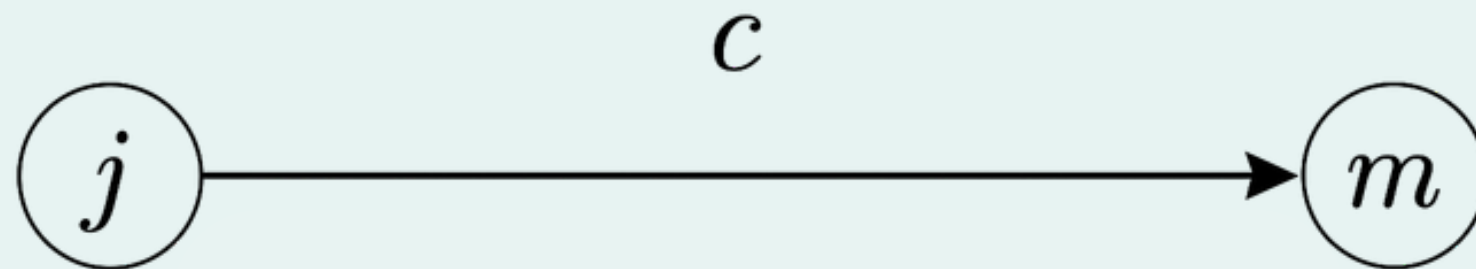
1. The direct edge ($j \rightarrow m$), with cost c .
2. The long path ($j \rightarrow k \rightarrow l \rightarrow m$), with cost g , but that value was already calculated and stored earlier.

So, when the algorithm needs the cost from j to m ,

- it doesn't recalculate the long path again;
- it retrieves the stored value from memory (the dynamic programming table).

Held-Karp Algorithm

Dynamic Programming for the Traveling Salesman Problem (Held-Karp)



How it works

In this case, we want the cost from $j \rightarrow m$.

There are two possible ways:

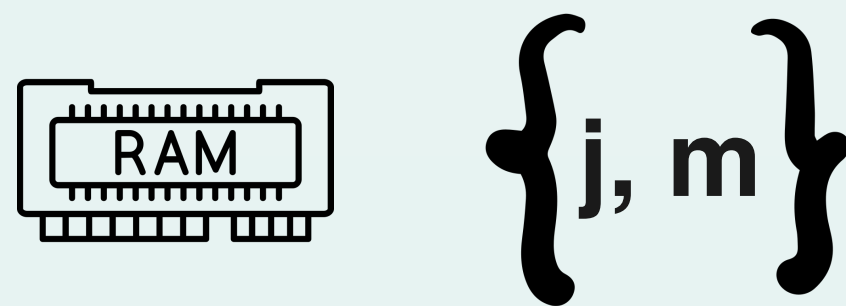
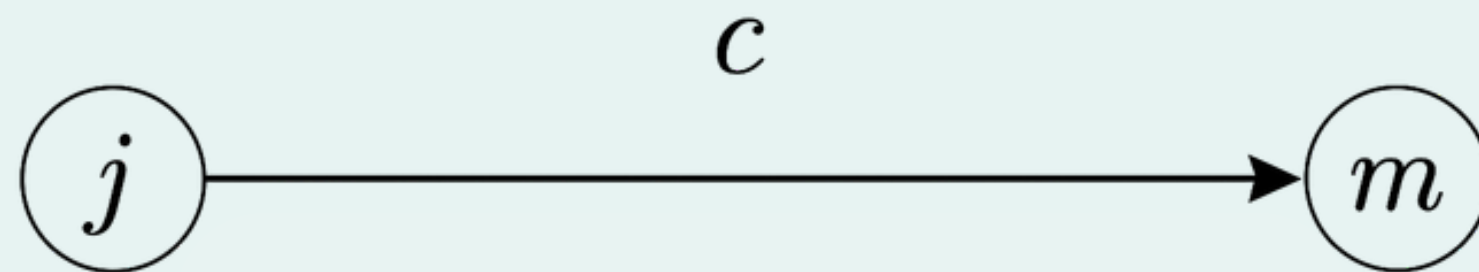
1. The direct edge ($j \rightarrow m$), with cost c .
2. The long path ($j \rightarrow k \rightarrow l \rightarrow m$), with cost g , but that value was already calculated and stored earlier.

So, when the algorithm needs the cost from j to m ,

- it doesn't recalculate the long path again;
- it retrieves the stored value from memory (the dynamic programming table).

Held-Karp Algorithm

Dynamic Programming for the Traveling Salesman Problem (Held-Karp)



How it works

In this case, we want the cost from $j \rightarrow m$.

There are two possible ways:

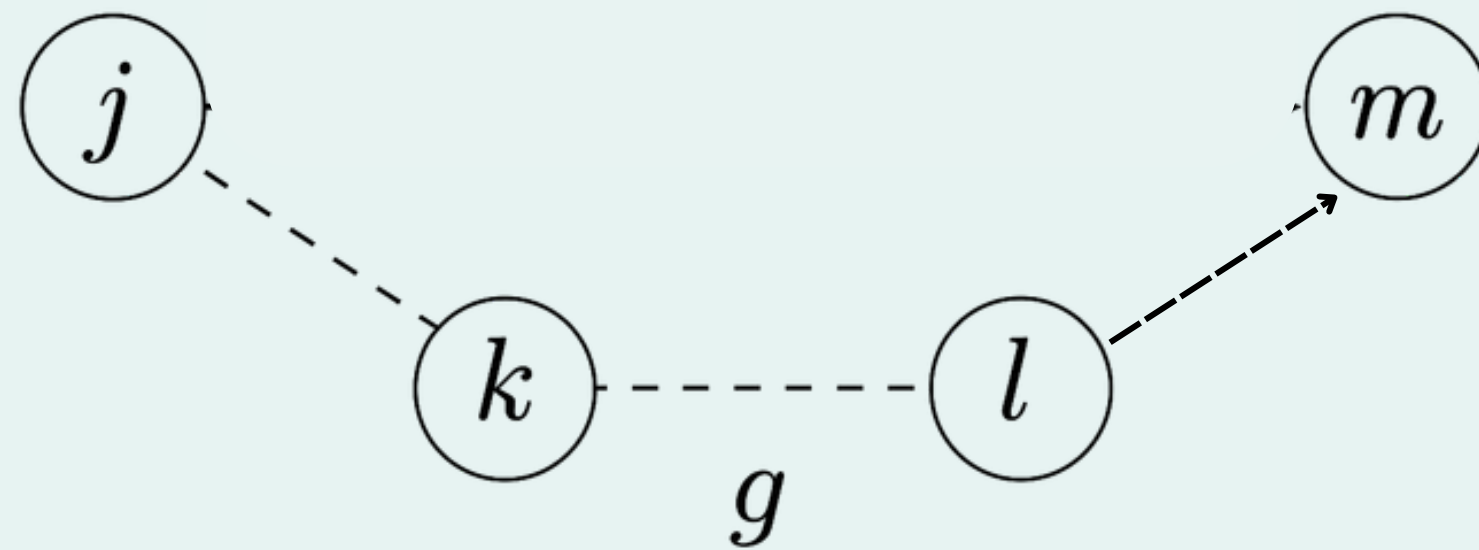
1. The direct edge ($j \rightarrow m$), with cost c .
2. The long path ($j \rightarrow k \rightarrow l \rightarrow m$), with cost g , but that value was already calculated and stored earlier.

So, when the algorithm needs the cost from j to m ,

- it doesn't recalculate the long path again;
- it retrieves the stored value from memory (the dynamic programming table).

Held-Karp Algorithm

Dynamic Programming for the Traveling Salesman Problem (Held-Karp)



How it works

In this case, we want the cost from $j \rightarrow m$.

There are two possible ways:

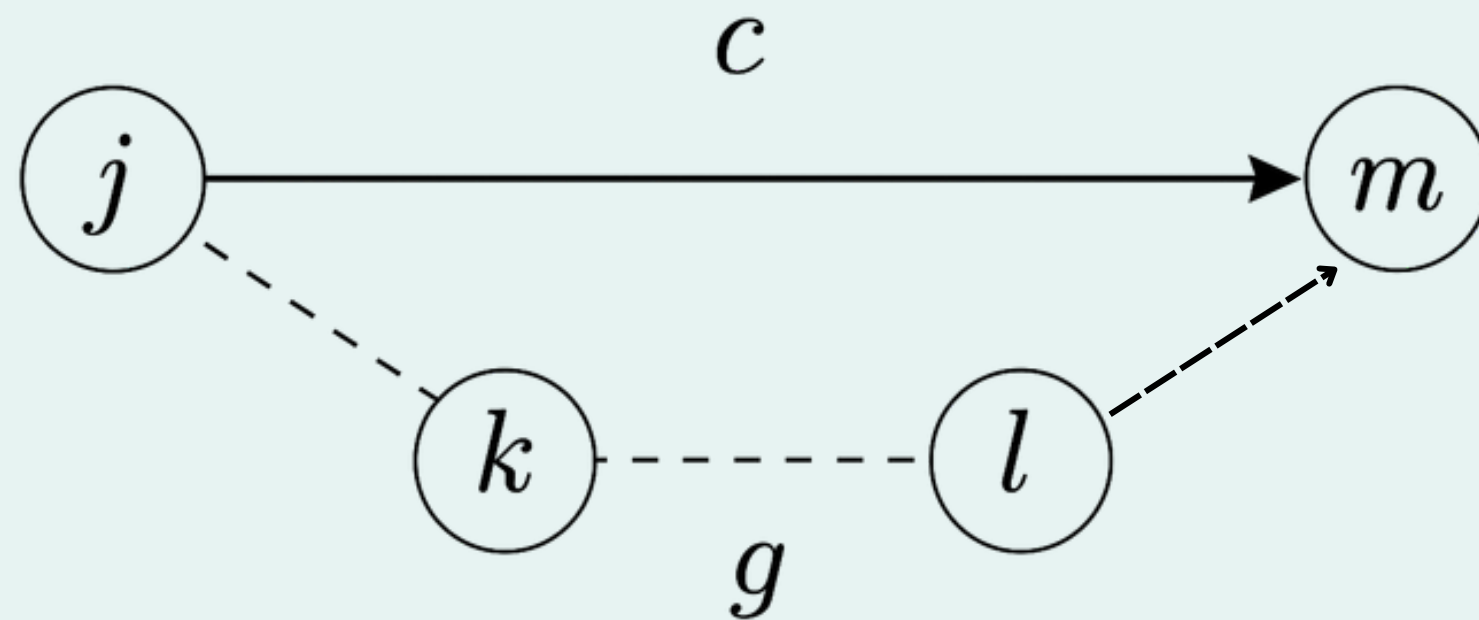
1. The direct edge ($j \rightarrow m$), with cost c .
2. The long path ($j \rightarrow k \rightarrow l \rightarrow m$), with cost g , but that value was already calculated and stored earlier.

So, when the algorithm needs the cost from j to m ,

- it doesn't recalculate the long path again;
- it retrieves the stored value from memory (the dynamic programming table).

Held-Karp Algorithm

Dynamic Programming for the Traveling Salesman Problem (Held-Karp)



How it works

In this case, we want the cost from $j \rightarrow m$.

There are two possible ways:

1. The direct edge ($j \rightarrow m$), with cost c .
2. The long path ($j \rightarrow k \rightarrow l \rightarrow m$), with cost g , but that value was already calculated and stored earlier.

So, when the algorithm needs the cost from j to m ,

- it doesn't recalculate the long path again;
- it retrieves the stored value from memory (the dynamic programming table).

Limitations

Data & API

- Single best offer per day: we only take the cheapest flight each day, ignoring alternatives (earlier departures, shorter durations, better connections).
- Sandbox data $\neq$ real-world data: Amadeus sandbox has limited routes and dummy pricing.

Modeling simplifications

- Objective = total flight cost only: doesn't consider the 'cost' of duration, layovers, comfort, or emissions.
 - Improvement: make it multi-objective: price + time + connections + CO₂, etc
- No non-flight costs: hotel, transport, visa fees, etc. are excluded.
 - Improvement: add fixed or per-day city costs and penalties.
- City = airport code: multi-airport cities (e.g. CDG/ORY) aren't grouped.
 - Improvement: treat metro areas as one node.
- Single passenger, economy class: no baggage or cabin differentiation.

